

DEFINITION

What oTTeVerse is building

oTTeVerse will and is building a modern Rust blockchain for games and metaverse economies. We treat assets as first-class objects, keep gameplay off the hot path, and settle ownership and value on a high-throughput L1.

We synthesize what already works in production — object-centric models like Sui, parallel execution like Solana and Aptos, hybrid on/off-chain game loops — rather than cloning any one chain.

Design goals

oTTeVerse will and is building toward a chain that games can actually run on:

- Sub-second finality for simple asset moves (equip, trade, craft).
- Tens to hundreds of thousands of TPS when transactions do not contend on the same objects.
- Near-zero fees for in-game micro-actions.
- True ownership of items, land, characters, and currencies that can move across titles.
- Gameplay that still feels like a game (60+ FPS, physics, AI) — not a wallet confirmation every click.
- One language for protocol and contracts: Rust.

Gameplay stays off-chain or in a WASM coprocessor. The chain is the ledger of record for the digital economy.

Layered architecture

oTTeVerse will and is building this stack, from clients down to storage:

Game clients (Unity / Unreal / Bevy / web WASM)

Wallets, marketplaces, social, studios

↓ SDKs, indexers, RPC, events

Application layer

Marketplaces · royalties · crafting · land · identity · native NFT / FT standards · escrow · tournaments

↓

Execution + VM

Object store · parallel scheduler · WASM (Rust contracts) · optional attested off-chain WASM coprocessor

↓

Consensus · data availability · networking

DAG + BFT fast path · object snapshots and erasure coding · gossip + turbine · QUIC

↓

Storage

Hot: object DB (Rocks / Sled / custom) Cold: DA / IPFS hashes

1. Object-centric state

oTTeVerse will and is building an object-centric state model. We do not store everything as "account + balance." We treat every asset as an object with:

- a globally unique ID
- an owner (address, another object, or shared)
- type + version
- capabilities (transfer, mutate, wrap)

A sword object can be owned by a character object owned by a player. Independent objects execute in parallel with no global lock. Shared objects — a marketplace pool, a raid boss — go through consensus. This matches how games already think about inventories.

2. Consensus and execution

oTTeVerse will and is building the validator, networking, and execution engine as a Rust workspace (Tokio, QUIC, blst / ed25519, custom object DB). There is no EVM unless we add it later as an optional sidecar.

- Fast path: single-owner object transactions skip full ordering; we certify and commit in about 100–400 ms.
- Slow path: shared objects use a DAG mempool (Narwhal-style) plus BFT (Bullshark / Mysticeti-class) so data dissemination is not the bottleneck.
- Parallel execution: the scheduler uses declared object IDs (explicit, like Solana accounts) plus ownership so non-overlapping transactions run concurrently. Optimistic STM (Block-STM style) is the fallback for shared state.
- Finality: deterministic, no long reorgs — required if items have real value.

3. Smart contracts in Rust

oTTeVerse will and is building two complementary runtimes so studios write Rust once and choose where it runs:

Runtime	Use	Why
On-chain WASM	Economy: mint, trade, royalties, staking, land deeds	Sandboxed, portable, Rust → wasm32
Native / BPF-like programs	Ultra-hot paths (order books, tick engines)	Solana-style performance
Off-chain WASM coprocessor	Combat resolution, physics, matchmaking, RNG with commit-reveal	Near-native speed; only hashes / outcomes land on-chain

Contracts express object types and capabilities, not raw storage slots. That keeps asset composition — a bag of items, nested NFTs — cheap and safe.

4. Hybrid game loop

oTTeVerse will and is building a hybrid loop that is non-negotiable. Players never wait on a block for a jump. A backend / sequencer cluster runs the same Rust WASM game handlers and posts periodic checkpoints or fraud / validity proofs.

Lives on-chain	Lives off-chain / coprocessor
Ownership, balances, marketplace fills	Input, rendering, physics
Crafting recipes that mint or burn assets	Frame-by-frame combat
Land deeds, rent, royalties	Chat, presence, pathfinding
Tournament prizes, escrow	Anti-cheat sensors (attest results)

5. Digital economy primitives

oTTeVerse will and is building these as native object types, not ad-hoc contracts:

- FT + NFT + dynamic NFT (stats that mutate without reminting).
- Composability: objects own objects (character owns loadout owns gems).
- Royalties / creator cuts enforced at transfer.
- Escrow + atomic swap for P2P and studio marketplaces.
- Fee market: priority fees + storage rent so abandoned items do not bloat state.
- In-game currency as a first-class coin object, optionally bridged.
- Identity: passkeys / zkLogin so a new player is not forced through seed phrases on day one.

Media (meshes, textures) stays off-chain. The object stores a content hash + URI.

6. Data, indexers, and studio APIs

oTTeVerse will and is building the data plane games actually query:

- Object DB + Merkle / verkle snapshots for light clients and rollbacks.
- A dedicated game indexer in Rust: inventory views, activity feeds, leaderboards, "what changed since last frame."
- Event bus (WebSocket / gRPC) so engines subscribe to their objects only.
- Oracles only where needed (fiat ramps, external randomness with VRF).

7. Modularity and interoperability

oTTeVerse will and is building consensus, execution, and data availability as separable layers so we can later:

- run an app-specific rollup for one title,
- use an external DA layer for cheap media commitments,
- bridge assets to Ethereum / Solana without rewriting the object model.

Bridges should move objects, not just wrapped ERC-721s.

8. Security model for valuable items

oTTeVerse will and is building protocol-level rules because games with real money attract duplication and admin keys:

- No silent mint: supply changes are typed object operations.
- Shared objects have explicit shared-vs-owned modes.
- Admin / studio keys are time-locked or DAO-gated for economy parameters.
- Deterministic WASM + attested coprocessor outputs; we never trust a single game server for mint.
- Rate limits and storage rent against spam inventories.

Rust crate map

oTTeVerse will and is building the node as a Rust workspace. This is the implementation sketch:

- node — validator binary
- consensus — DAG + BFT
- execution — object store + scheduler
- vm-wasm — contract runtime
- objects — NFT / FT / land types
- p2p — QUIC gossip
- rpc / indexer — studio and engine queries
- sdk — Rust + FFI for engines
- coprocessor — optional verifiable game runtime

Contracts use `#[object]` macros and capability types, tested against an in-memory object store — the same pattern as Anchor / Sui Move tests, but in Rust.

What we will not do

oTTeVerse will and is building by refusing the designs that make game chains unplayable:

- Put every combat tick on L1.
- Use a global account model as the only state shape.
- Make players pay Ethereum-mainnet fees for a potion.
- Hide the studio as an unbounded mint authority.
- Invent a new contract language when Rust + WASM already exists.

This stack is modern because it matches how games actually work: many independent objects moving at once, a thin settlement layer for the economy, and Rust from node to contract to (optionally) game logic. Sui-style objects plus Solana-class parallelism plus a WASM coprocessor is the combination that currently fits metaverse digital economies best.